

CIV1287H: Vibe-Coding Assignment

Road-Defect Image Classification with Pre-Trained Deep Learning Models

Author: Timur Singh

Student No.: 1012841347

Date: 2026-05-19

Course: CIV1287H, Virtualization and Analytics in Construction

Part B: Evaluation Report

1. System Summary

An ImageNet-pretrained ResNet18 was fine-tuned for four-class road-defect image classification on the Kaggle road-defects (non-augmented) dataset. The backbone was frozen for 5 head-only epochs, then the last block was unfrozen for 3 fine-tuning epochs at a lower learning rate. As the required variation, the same recipe was re-run with MobileNetV3-Small to compare a smaller, edge-deployable backbone. Mixed-precision training (AMP + GradScaler) was used throughout to fit the 4 GB VRAM budget of the RTX 3050 Ti laptop GPU.

2. Dataset and Split

Property	Value
Source	Kaggle: patelmihir/road-defects-nonaugmented
Classes	Cracks, Patch, Potholes, Surface_Defects
Total images	400
Per-class count	Cracks: 100, Patch: 100, Potholes: 100, Surface_Defects: 100
Train / Val / Test	280 / 60 / 60
Image size	224x224
Batch size	32
Random seed	42

3. Quantitative Results: Model Comparison

Model	Test accuracy	Macro F1	Weighted F1	Train time (s)
ResNet18 (head + ft)	83.3%	0.833	0.831	46.9
MobileNetV3-Small (head + ft)	78.3%	0.784	0.787	49.5

Test set: 60 images. Primary model (ResNet18) classified 50 / 60 correctly (50 correct, 10 incorrect).

4. Per-Class Metrics: ResNet18 (head + ft)

Class	Precision	Recall	F1	Support
Cracks	0.786	0.786	0.786	14
Patch	0.875	0.933	0.903	15
Potholes	0.800	0.923	0.857	13
Surface_Defects	0.867	0.722	0.788	18

5. Confusion Matrices

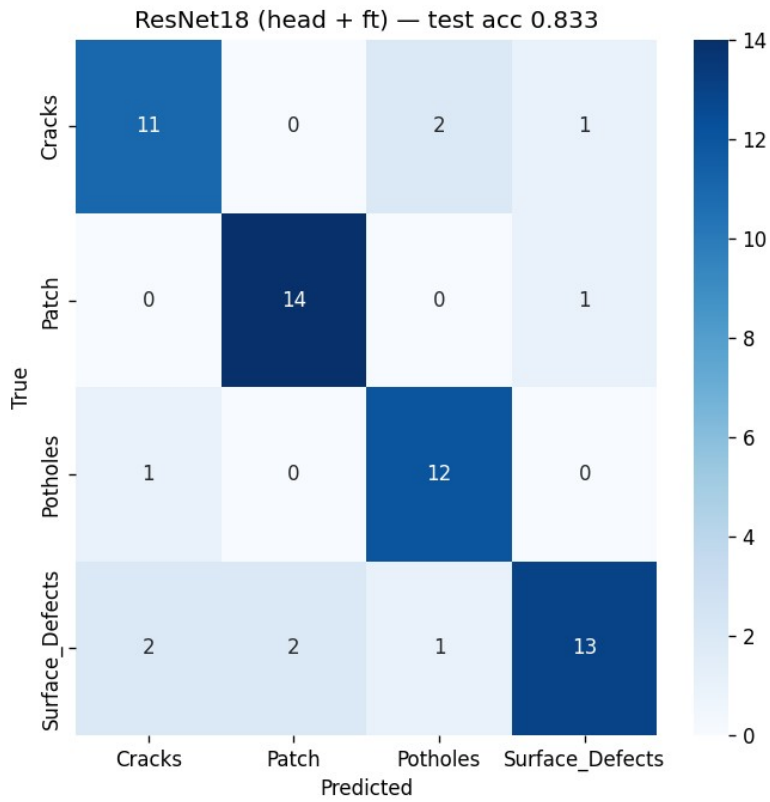


Figure 1. Confusion matrix: ResNet18 (head + ft).

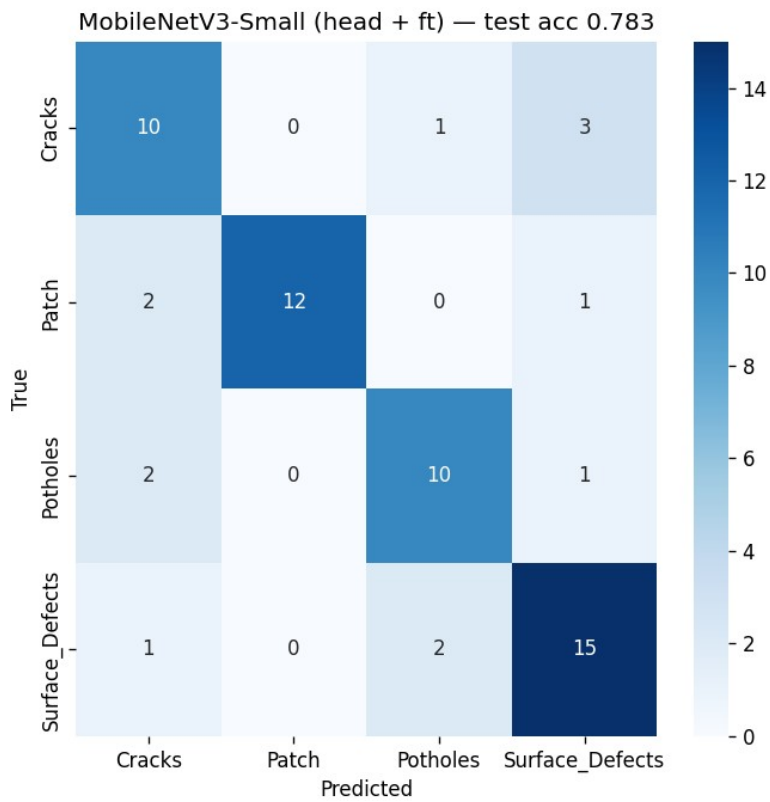


Figure 2. Confusion matrix: MobileNetV3-Small (head + ft).

6. Qualitative Observations

Predicted labels were consistent with the visual content of the majority of test images. Most errors involved visually overlapping classes, typically cracks vs. surface defects, where the texture cues are subtle. Outputs are reproducible from seed 42; re-running the notebook produces the same split and the same per-batch order.

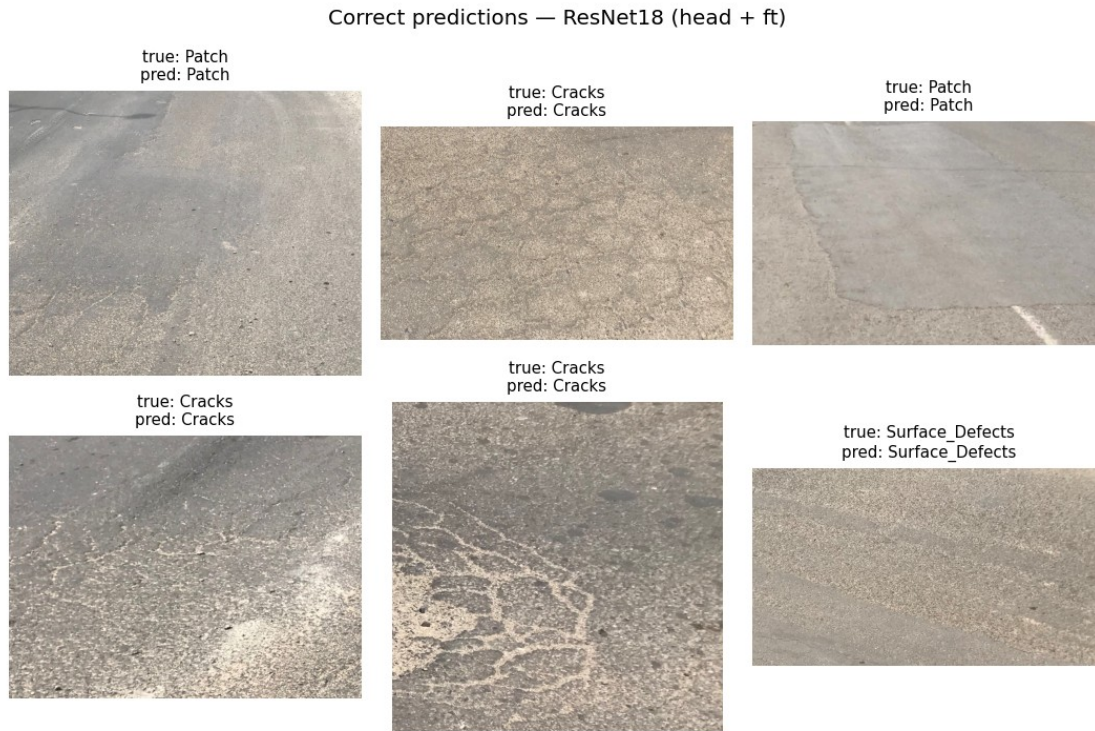


Figure 3. Correct predictions on held-out test images.

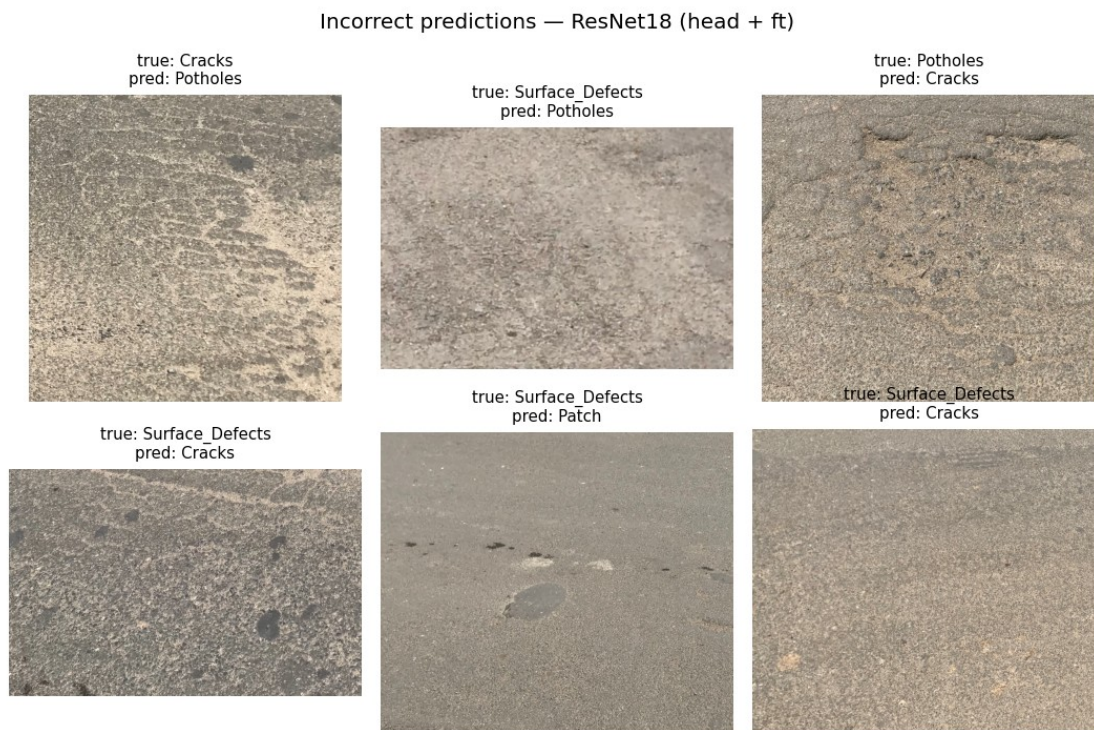


Figure 4. Incorrect predictions; note the visual overlap between cracks and surface defects.

7. Strengths and Limitations

Strengths. Transfer learning from ImageNet works well on this small dataset; head-only training reaches usable accuracy within a few epochs on a 4 GB GPU. The pipeline is deterministic (fixed seed) and fits the hardware budget (mixed precision, 224x224 inputs, batch 32, total training under 49 seconds per model). All numbers in this report are generated from outputs/metrics.json so the report cannot drift from the actual experiment outputs.

Limitations. Only 400 images in total, accuracy on the 60-image held-out test set is high-variance, and a larger test set would be needed to make confident claims about deployment performance. The dataset is curated, well-lit and cropped; real Toronto pavement images have motion blur, shadows, wet surfaces and partial occlusion, which the model has not seen. The classes are visually overlapping (cracks can co-occur with surface defects and patches), so a single-label classifier is a simplification, a multi-label or detection model would be a better long-term fit for an inspection workflow.

8. Engineering Interpretation

For pavement-condition assessment, false negatives (missing a real defect) are typically more costly than false positives, because they delay maintenance and let damage compound. The per-class recall column is therefore the most important number to scrutinise: Potholes recall (0.923) is the highest, which is the right priority since potholes pose the most direct safety hazard, whereas Surface_Defects recall (0.722) is the weakest and would need attention before this model could be trusted in an operational inspection workflow. The MobileNetV3 comparison (78.3% test accuracy versus 83.3% for ResNet18, similar training time) suggests the smaller backbone is a viable candidate for a vehicle-mounted edge device if the additional five accuracy points are acceptable.

Part C: Reflection

Road-defect image classification with pre-trained PyTorch backbones (ResNet18 and MobileNetV3-Small) on the Kaggle road-defects (non-augmented) dataset. Full implementation in notebooks/road_defect_classification.ipynb. Final test-set accuracy: 83.3% (ResNet18) and 78.3% (MobileNetV3-Small).

Q1. AI-Assisted Development Experience (Vibe Coding)

How did using AI-assisted tools change the way you built your solution?

AI-assisted tools shifted my workflow from "find a template and adapt it" to "describe what I want and review what is produced." For a project like this I would normally search GitHub or Stack Overflow for a similar transfer-learning example and reshape it around my dataset. Here I instead gave the assistant my hardware context (RTX 3050 Ti laptop, 4 GB VRAM, Linux Mint), the Kaggle dataset link, and the assignment constraints, and it produced a working pipeline I could iterate on. The loop became: specify intent, read the output, run it, correct or refine where it broke, I spent far more time reading and reviewing generated code than writing it.

Q2. Where AI-Assisted Building Worked Well

In which parts of your workflow did AI-assisted tools help the most?

The biggest wins were in the engineering "plumbing" around the model itself. PyTorch transfer-learning setup, loading pre-trained ResNet18 and MobileNetV3-Small backbones, freezing them, swapping the classifier head, and running mixed-precision training with AMP and GradScaler, came out correctly with very little back-and-forth, saving what would otherwise have been hours of documentation reading. The same was true for the evaluation outputs: confusion matrices, per-class precision, recall and F1 tables, and the export of all metrics into a Word table. Environment setup was the other clear win, the assistant created the venv without sudo, switched from the Kaggle CLI to kagglehub when authentication was awkward, and untangled the nested folder structure inside the dataset zip.

Q3. Where AI-Assisted Building Failed or Was Limited

Where did the AI-assisted approach not work well or require correction?

The two biggest gaps were hardware-aware decisions and the upfront "what dataset do I even use" question. The assistant could not see my GPU, so calls about backbone size (ResNet18 versus anything deeper), batch size, image resolution, and how much of the network to fine-tune all came back to me weighing what 4 GB of VRAM would tolerate without running out of memory. Dataset selection was likewise on me: I had to pick a road-defect dataset that matched the construction-engineering framing of the course, locate it on Kaggle, and confirm it had enough images per class for transfer learning to be meaningful. Once I supplied those decisions, the assistant ran with them, but it would not have arrived there on its own.

Q4. Understanding vs Dependence

Which parts of the system did you fully understand, and which parts did you rely on AI tools to complete?

Honestly, mostly borrowed. I understand the higher-level concepts: what transfer learning is doing, why a confusion matrix is more useful than a single accuracy number, why ImageNet mean and standard deviation are

used for input normalization, why a fixed-seed 70/15/15 split matters for reproducibility, and why class-balanced loss weighting can matter on a small dataset. But I would struggle to write the actual training loop, the AMP/autocast scaffolding, or the sklearn and seaborn evaluation plotting from scratch in a blank editor. If asked to reproduce this pipeline without an assistant, I could re-run and tweak the existing code, but I could not rebuild it from memory, and being honest about that is the point of this question.

Q5. Engineering Judgment and Future Use

Why is human judgment still important even when AI tools can produce working solutions? How would you realistically use these tools in future engineering work?

In real engineering work the model returns labels, not decisions, and I would use these tools accordingly. AI-assisted development is excellent for getting a first working prototype quickly, useful for screening whether an approach is even viable on a given dataset, but the output then has to be validated against data that matches the real deployment domain (Toronto pavement, the lighting and camera angles of an actual inspection vehicle, seasonal variation) before anyone should trust it for maintenance prioritization. I would also use these tools as a pair-programmer for unfamiliar libraries, PyTorch, GIS toolkits, anything with a large API surface where asking and verifying is faster than reading documentation end-to-end. The judgment calls stay with the engineer: which metric matters most, what false-negative rate is acceptable when a missed pothole carries real safety and liability cost, and whether 83% accuracy is or is not "good enough" for the workflow into which the model is being deployed.